Tema 1

8 notes · Exported 2026-10-23

Contents

1. 0 Presentación 3
1. 1 Entornos de un Solo Agente 5
1.2 Espacios de Estados 7
1.3 Búsquedas con árbol 10
1.3 Búsquedas con Árbol (Continuación) 12
1.3.2 Búsquedas con Árbol 15
1.4 Búsquedas con marcha atrás 18
1.5 Heurísticos Bases de Datos de Patrones 19

1. 0 Presentación

2026-09-10

Mandow Lorenzo: Despacho 3.2.38 --- lmandow@uma.es

Introducción:

La IA se divide en:

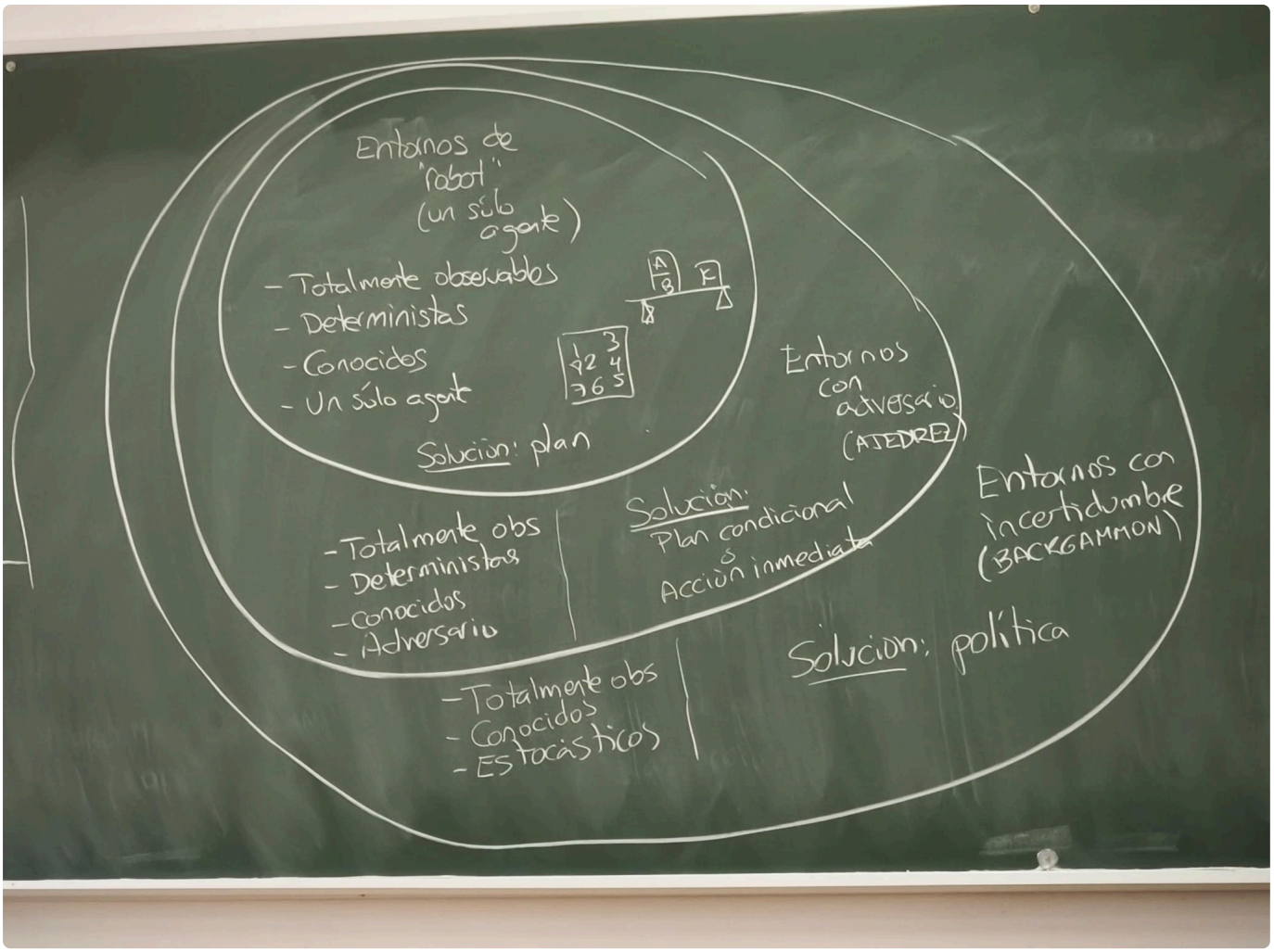
- Aprendizaje (predicción, inducción)
- Resolución de problemas (deducción) ⋆

IA simbólica

Se basa en la **Hipótesis del Sistema de Símbolos Físicos** (Newell y Simon, 1976)

- Un SSF posee los medios necesarios y suficientes para acciones inteligentes generales.
- Un SSF posee:
- Un conjunto de expresiones simbólicas
- Unos procesos que las manipulan
- Los seres humanos y los ordenadores pueden usar el SSF

Hipótesis de la Búsqueda Heurística
Los problemas se resuelven buscando de forma selectiva (heurística) en un espacio de problema (representación del problema).


Entornos de "robot" (un sólo agente)
- Totalmente observables
- Deterministas
- Conocidos
- Un sólo agente
Solución: plan
Entornos con adversario (AJEDREZ)
- Totalmente obs
- Deterministas
- Conocidos
- Adversario
Solución: Plan condicional ó Acción inmediata
Entornos con incertidumbre (BACKGAMMON)
- Totalmente obs
- Conocidos
- Estocásticos
Solución: política

1. 1 Entornos de un Solo Agente

2026-09-10

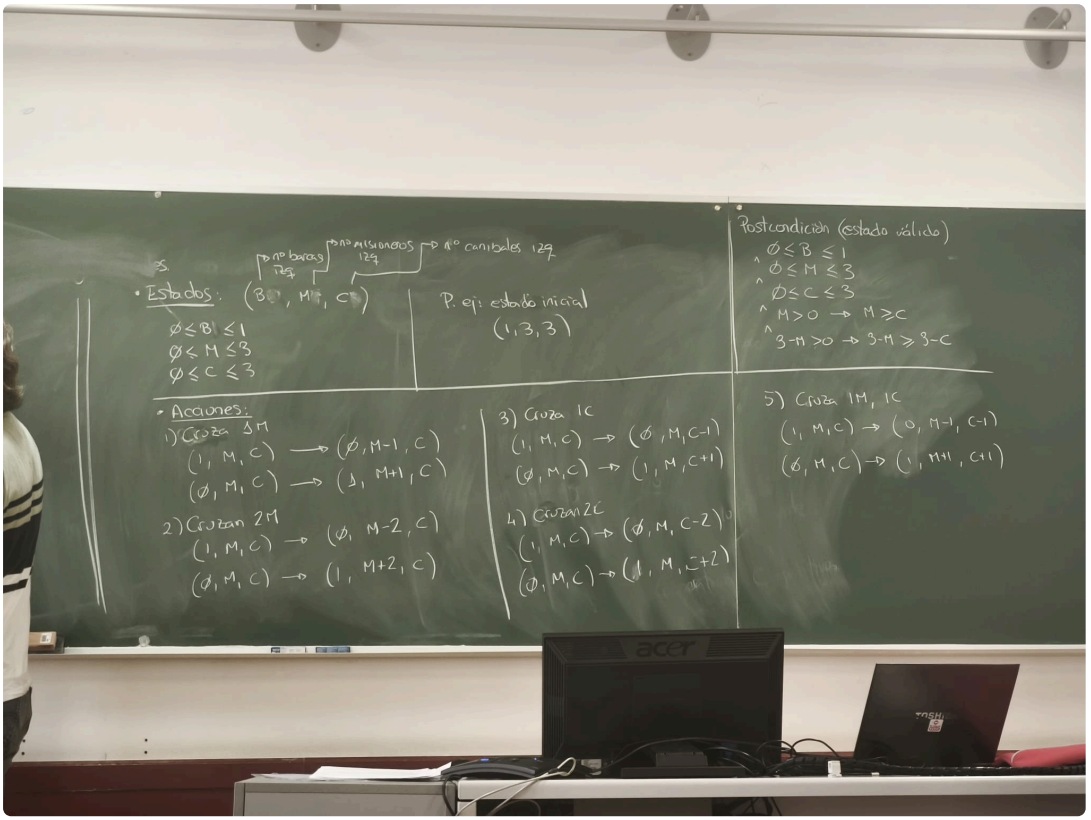
1. Representación: Espacios de Estados

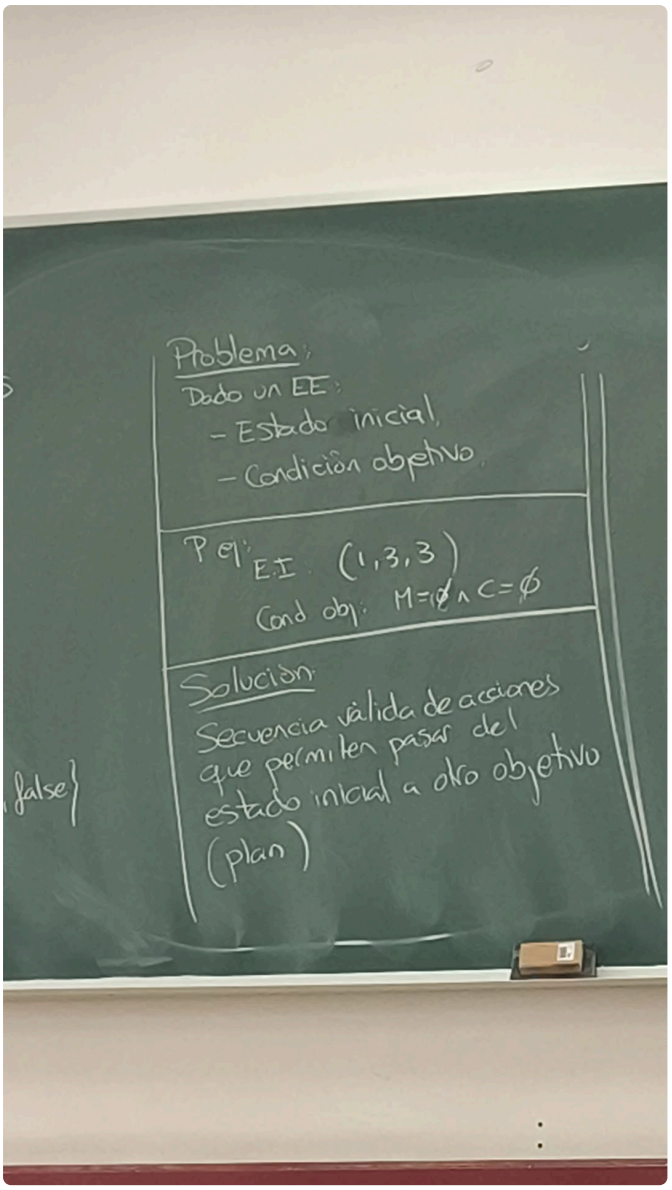
Representación formal de una parte relevante del mundo.

Un EE se compone de:

- Conjunto de estados (E) (Posibles situaciones del mundo)
- Conjunto de acciones (cómo pasar de un estado a otro)
  - Precondición: E -> {true, false}
  - Efectos: E -> E

Ejemplo: Problema de los Misioneros y los caníbales




Problema:
Dado un EE:
- Estado inicial,
- Condición objetivo,
P ej:
E.I (1,3,3)
Cond obj: M=∅ ∧ C=∅
Solución:
Secuencia válida de acciones
que permiten pasar del
estado inicial a otro objetivo
(plan)
false}

1.2 Espacios de Estados

2026-09-15

Búsquedas con Árbol

- Amplitud
- Dijkstra
- A*

Un espacio de estados se compone de:

- Conjunto de Estados (Posibles situaciones)
- Conjunto de acciones (Pasar de un estado a otro)
  - Precondición
  - Efectos

Puzle 8

El puzle 8 está formado por una cuadrícula de 3×3 con ocho piezas numeradas y un hueco. Un movimiento consiste en deslizar hacia el hueco una pieza adyacente.

El estado inicial considerado es:

3	4	2
0	1	5
7	6	8

El valor 0 representa el hueco.

Representación de un estado

Cada estado se representa mediante una matriz de enteros de tamaño 3×3:

$$E = \begin{array}{|c|c|c|} \hline e_{0,0} & e_{0,1} & e_{0,2} \\ \hline e_{1,0} & e_{1,1} & e_{1,2} \\ \hline e_{2,0} & e_{2,1} & e_{2,2} \\ \hline \end{array}$$

La posición del hueco se denota por (F_i, C_j), donde F_i es la fila y C_j es la columna. Para el estado inicial:

$$E_0 = \begin{array}{|c|c|c|}\hline 3 & 4 & 2 \\ \hline 0 & 1 & 5 \\ \hline 7 & 6 & 8 \\ \hline \end{array}, \qquad (F_i, C_j) = (1, 0).$$

Acciones

Las acciones consisten en mover el hueco en una de las cuatro direcciones posibles. En cada caso se intercambia el hueco con la pieza adyacente mediante una operación `swap`.

| Acción | Precondición | Efecto | Coste |

| Mover el hueco hacia arriba | $F_i > 0$ | $\text{swap}((F_i, C_j), (F_i - 1, C_j))$ | 1 |

| Mover el hueco hacia abajo | $F_i < 2$ | $\text{swap}((F_i, C_j), (F_i + 1, C_j))$ | 1 |

| Mover el hueco hacia la derecha | $C_j < 2$ | $\text{swap}((F_i, C_j), (F_i, C_j + 1))$ | 1 |

| Mover el hueco hacia la izquierda | $C_j > 0$ | $\text{swap}((F_i, C_j), (F_i, C_j - 1))$ | 1 |

Por tanto, una acción solo se puede aplicar si se cumple su precondición. Después de aplicar la acción, el hueco intercambia su posición con la pieza contigua correspondiente.

Orden del token -> d(t_1, <t_1, t_2, ..., t_k)

d(2, <1,5,2>) = 1 -> Porque si ordeno <1,5,2> me queda <1,2,5> y el número 2 está en la posición 1.

d(5, <1,5,2>) = 2

Rankp me dice el orden de una tupla en una lista ordenada.

Rankp de <2,0,1> es 4 porque: {<0,1,2>, <0,2,1>, <1, 0,2>, <1,2,0>, <2,0,1>, ...} si ordeno las posibles combinaciones se me queda en la 4 posición.

La fórmula es:

$$rankp(< t_1, t_2, \ldots, t_k) = d(t_1, < t_1, t_2, \ldots, t_k >) * (k-1)! + rankp(< t_2, \ldots, t_k >)$$

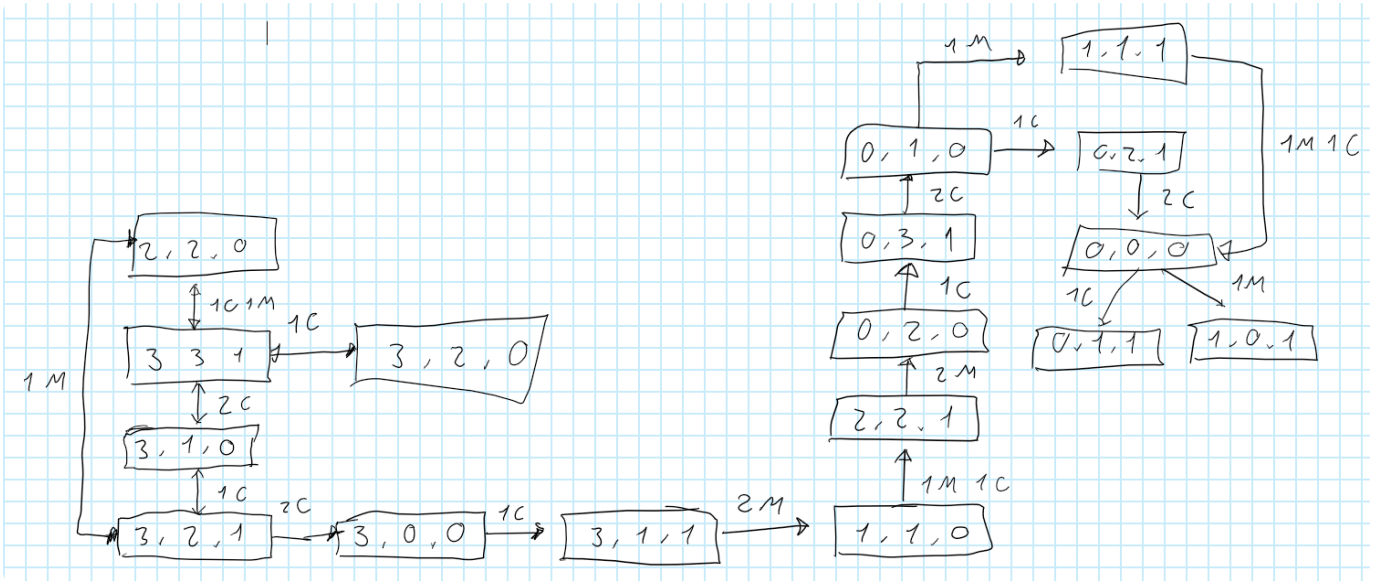
$$rankp(< t_1 >) = 0$$

rankp(<2,0,1>) = d(2, <2,0,1>) *2!* + *rankp(<0,1>) = d(2, <2,0,1>)* 2! + d(0, <0,1>) * 1! + 0 = 4

Unrankp([0,1,2], 5) -> NO CAE

Un Espacio de Estados define implícitamente un grafo:

- Estados == Nodos
- Acciones == Arcos



Estrategias de búsqueda

Estrategias irrevocables

Estas estrategias exploran un único camino:

- Búsqueda local
- Algoritmo genéticos

Estrategias tentativas

Estas estrategias exploran sistemáticamente el grafo:

- Con árbol: Guardan todos los caminos interesantes.
- Con retroceso: Solo guardan el camino actual.

1.3 Búsquedas con árbol

2026-09-22

Lo que caracteriza a cada método de búsqueda es:

1. Criterio de selección de **ABIERTOS**.
2. Qué hacer cuando se encuentran 2 caminos al mismo nodo.
3. Criterio de terminación.

Método de búsqueda de **Amplitud**:

1. Abiertos nuevos -> Al final de la cola
2. Nos quedamos con el antiguo
3. Cuando se genera el estado objetivo / Cuando se selecciona para expansión el estado objetivo.

Propiedades de los algoritmos

Las propiedades de los distintos algoritmos/métodos de búsquedas con árbol son:

1. Completo: Si hay solución, la encuentra (en un número finito de pasos).
   - Ej: Amplitud es completo porque aunque el árbol sea infinito encuentra la solución porque esa solución tiene que estar a una profundidad finita.
2. Admisible: Si hay solución, encuentra la óptima.
   - Ej: Amplitud no lo es, en general, porque ni siquiera tiene en cuenta el coste. Si todos los arcos tienen el mismo coste (el coste es la profundidad), ahí si que encuentra la solución óptima.
3. Consumo de memoria (nº de nodos almacenados en el árbol) y tiempo (nº de iteraciones (selecciones de abiertos)).
   - Un árbol infinito con factor de ramificación b (branch) con solución a profundidad d(depth) tiene un coste de memoria de $O(b^d)$
4. Malla cuadrada de 4 vecinos.

Dijkstra

Algoritmo de búsqueda a ciegas.

1. Selección de ABIERTOS: Menor valor de g(n).
2. 2 caminos al mismo nodo: Conservamos el de menor g(n).
3. Criterio de terminación: Cuando se **selecciona** el objetivo (con el algoritmo de amplitud también se podía acabar cuando se seleccionaba para expansión el nodo objetivo, ahora no porque puedes encontrar un

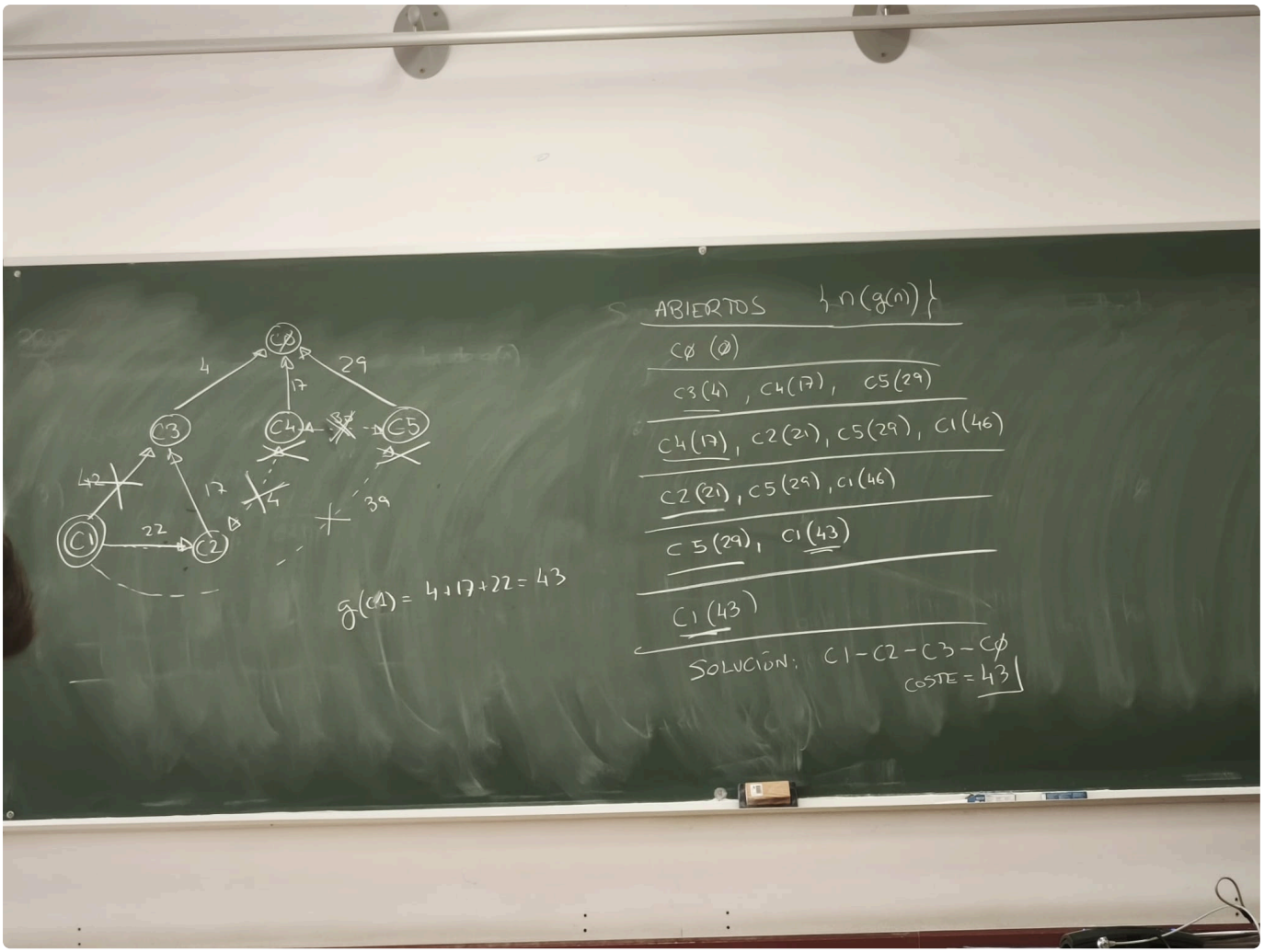
atajo).

Suposiciones:

1. Cada arco del grafo (n, n') tiene un coste c(n,n') >= 0
2. El coste de un camino es la suma de los costes de los arcos.
3. Caminos infinitos tienen infinitos. $\forall (n, n') c(n, n') \geq \varepsilon > 0$

Notación:

- g(n) = Coste de camino guardado en el árbol desde S (raíz) hasta n.



A*

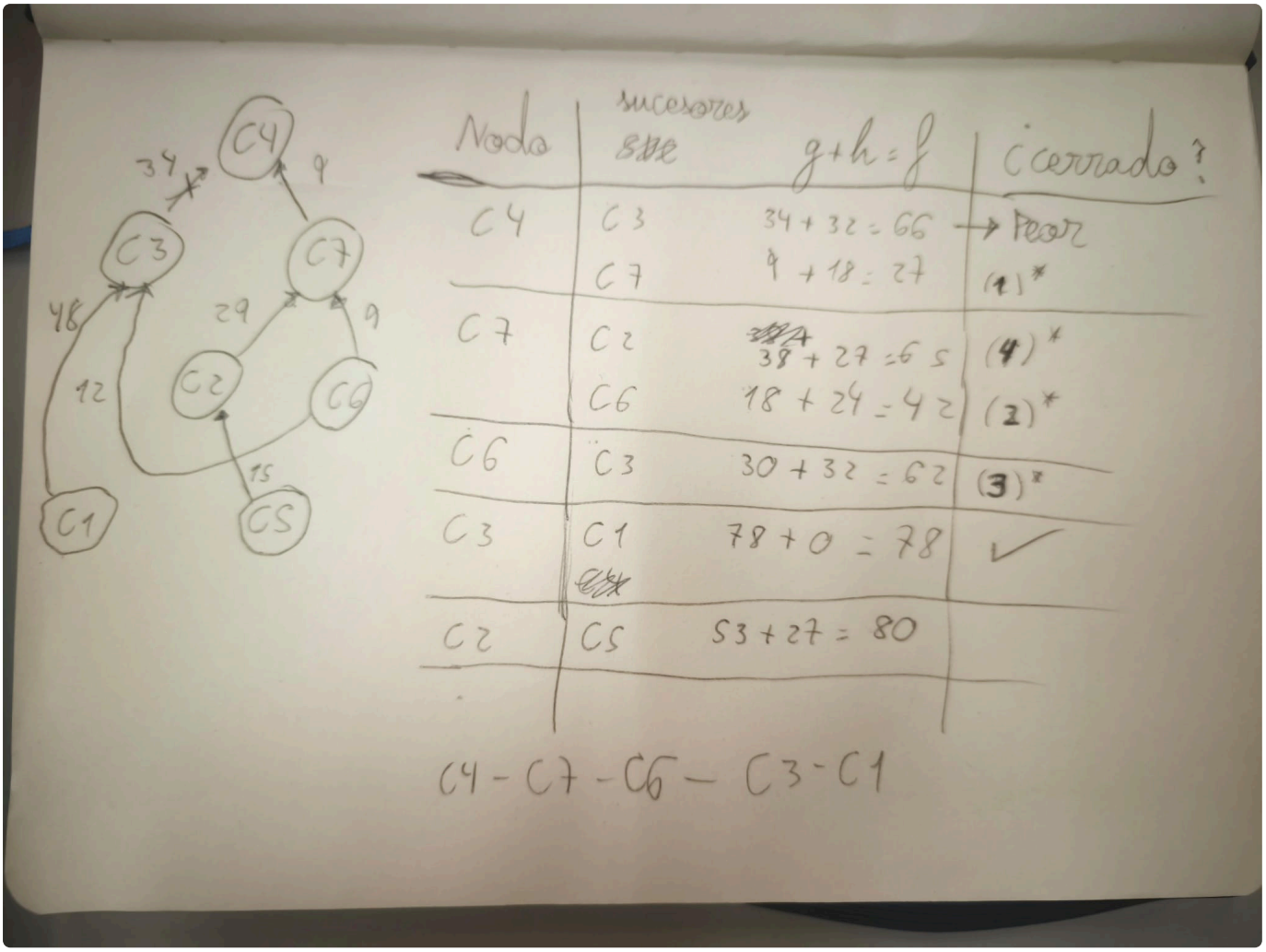
Algoritmo de búsqueda Heurístico: Conocimiento propio del problema para guiar la respuesta.

1.3 Búsquedas con Árbol (Continuación)

2026-09-25

El algoritmo A* se basa en:

1. Selección de abiertos:
   - menor $f(n) = g(n) + h(n)$
2. 2 caminos al mismo nodo:
   - Si el nuevo camino a n' es mejor, redirigir el puntero de n' a ese camino.
   - Si n' estaba cerrado, lo devolvemos a abierto.
   - Esto hace que podamos expandir el mismo nodo varias veces en A*.
3. Terminación:
   - Cuando seleccionamos el objetivo.



Propiedades de A*

$f(n) = g(n) + h(n)$

- $g(n) =$ Coste en el árbol hasta n
- $h(n) =$ Estimación del coste n al objetivo
- $f(n) =$ Estimación de coste de una solución que pasa por n

$c^* =$ Coste de la solución óptima
$g^*(n) =$ Coste óptimo real desde S hasta n. (S es el nodo raíz)
Normalmente $g(n) \geq g^*(n)$

$h^*(n) =$ Coste óptimo real desde n al objetivo.

$f^*(n) = g^*(n) + h^*(n)$

Suposiciones

1. $\forall n,\ \ h(n) \leq h^*(n)$ -> Optimista o admisible
2. Caminos infinitos tienen costes infinitos.
3. El nº de sucesores de un nodo es finito.

Propiedad 1

Si $h(n)$ es optimista, A^* es admisible.

LEMA 1: A^* termina siempre en grafos finitos.

LEMA 2: Sea $P^* = n_0, n_1, \ldots, n_k$ el camino solución óptimo: P^* nunca se poda.

LEMA 3: A^* termina siempre incluso en grafos infinitos.

LEMA 4: A^* no puede terminar con una solución subóptima.

Propiedad 2

Si h2 está más informado que h1, entonces si A* con h2 expande n, también lo hará con h1.

Un heurístico es Monótono si cuando hago un movimiento (rellenar)

Comprobar si el heurístico h2 es monótono (distancia euclidea (creo, revisar si es es o es la manhatan)a la posición final)

Propiedad 3:

Si h(n) es monótono, cuando A *selecciona un nodo para expanción, g(n)* = *g*(n) (Expandirá cada nodo a lo sumo 1 vez). -> Es mejor hacerlo con A *que con Dijstra porque A* aunque suela ser mejor, si expande los nodos varias veces puede tardar más.

EJERCICIO

Red de carreteras densa con p nodos por unidad de superficie.

Deseamos encontrar un camino óptimo entre A(0,0) y Z(0, a).

Se utiliza distancia euclídea como heurístico.
La distancia óptima entre A y Z es C*=C.

Calcular la cota superior nº nodos por Dijkstra y por A*

1.3.2 Búsquedas con Árbol

2026-09-28

Propiedades de A*

Propiedad 1

Si $h(n)$ es optimista, A^* es admisible.

LEMA 1: A^* termina siempre en grafos finitos.

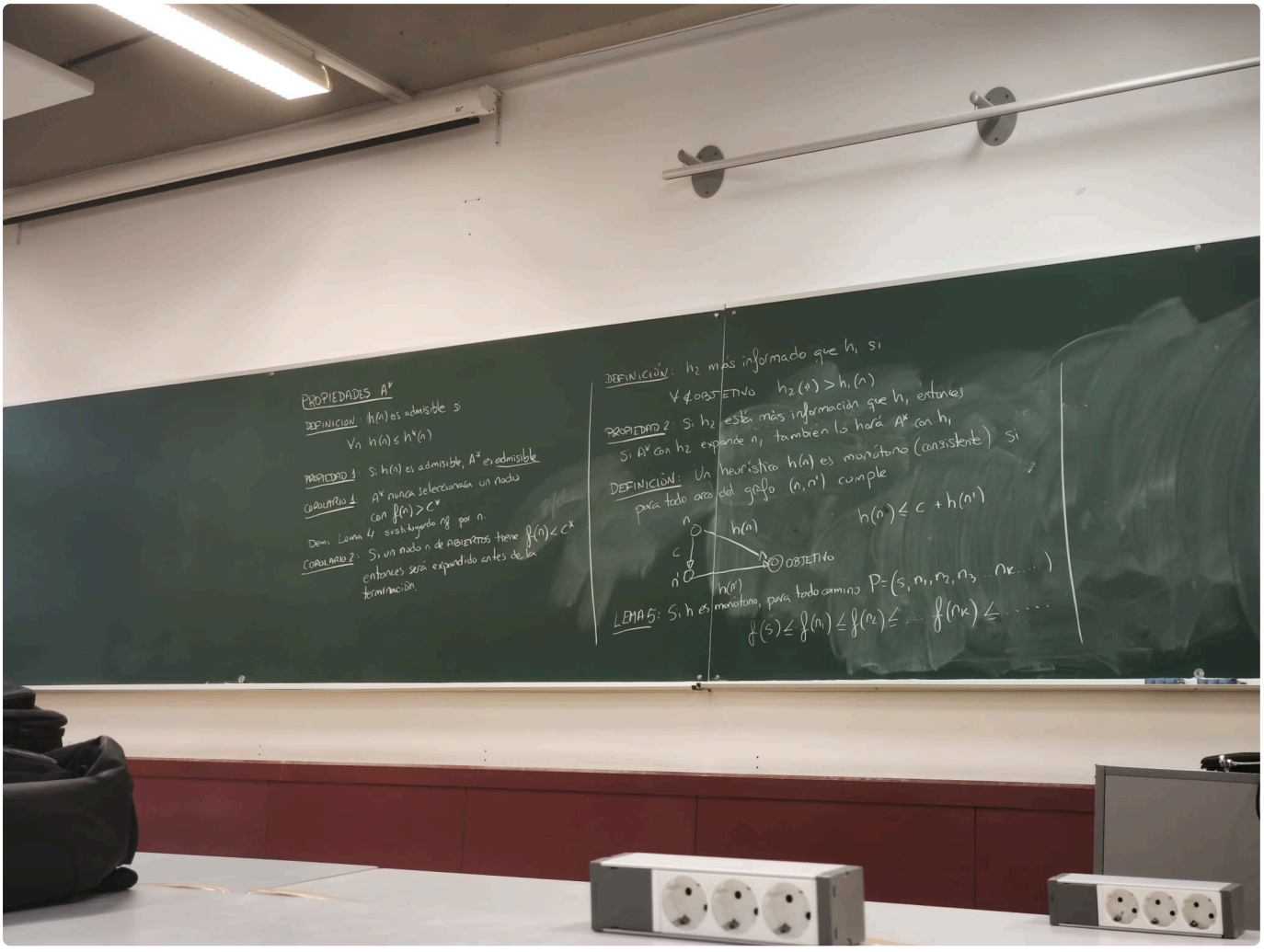
LEMA 2: Sea $P^* = n_0, n_1, \ldots, n_k$ el camino solución óptimo: P^* nunca se poda.

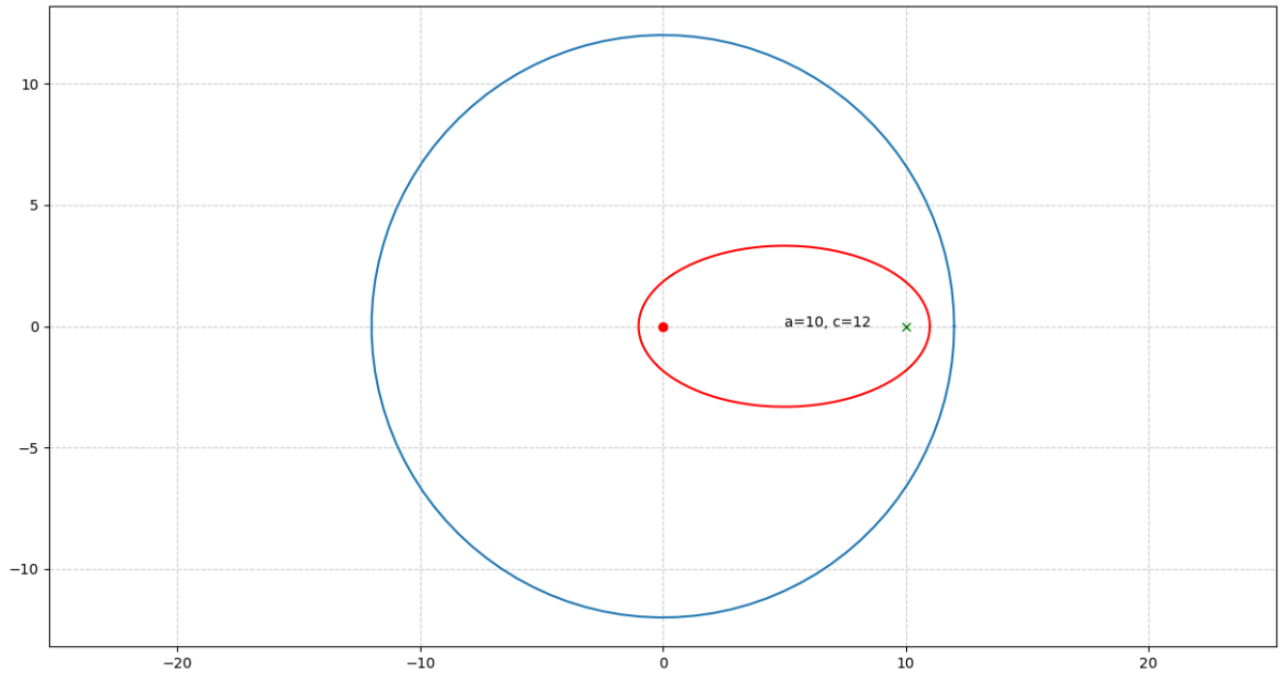
LEMA 3: A^* termina siempre incluso en grafos infinitos.

LEMA 4: A^* no puede terminar con una solución subóptima.

Propiedad 2

Si h2 está más informado que h1, entonces si A* con h2 expande n, también lo hará con h1.





El profe hizo una demostración en la pizarra que concluía en que para unos datos dados, siempre A *visita* <= *cantidad de nodos que Dijkstra (siendo la elipse roja la zona de búsqueda de A* y la zona azul la zona de búsqueda de Dijkstra).

A* suele ser mejor pero como a veces re-visita un nodo varias veces, en algunos casos puede ser peor.

Un heurístico es monótono, también llamado consistente, si para cada movimiento `n → n'` se cumple:

$$h(n) \leq c(n, n') + h(n')$$

donde `c(n,n')` es el coste del movimiento.

Si cada movimiento cuesta 1:

$$h(n) \leq 1 + h(n')$$

Equivalentemente:

$$f(n) \leq f(n')$$

Es decir, el valor de `f` no disminuye al avanzar por un camino.

Si `h` es monótono y A* trabaja sobre un grafo, cuando selecciona un estado para expandir se cumple:

$$g(n) = g^*(n)$$

Por tanto, cada estado se expande como máximo una vez y no es necesario reabrirlo posteriormente.

Distancia Manhattan

Si el movimiento es horizontal o vertical y cuesta 1, la distancia Manhattan es:

$$h_M(n) = |x - x^*| + |y - y^*|$$

Es monótona porque un movimiento cambia la distancia Manhattan como máximo en 1:

$$h_M(n) \leq 1 + h_M(n')$$

Propiedad 3:

Si h(n) es monótono, cuando A *selecciona un nodo para expansión, g(n)* = *g*(n) (Expandirá cada nodo a lo sumo 1 vez). -> Es mejor hacerlo con A *que con Dijsktra porque A* aunque suela ser mejor, si expande los nodos varias veces puede tardar más.

1.4 Búsquedas con marcha atrás

2026-09-29

Propiedades de Backtracking:

Completo:

En general no. Sí en grafos acíclicos dirigidos o árboles finitos.

Admisible:

No.

Consumo tiempo / memoria

1.5 Heurísticos Bases de Datos de Patrones

2026-10-08